Hackathon Q&A pack

Questions a CXO panel and technical judges are likely to ask about Zylo Reach, with answers. Zylo Reach finds the right creators to promote a product or collection, shows what they charge and the best plan for a budget, and explains every decision. All creator and sales data in this build is synthetic.

Section	What it covers
1. CXO and business	Value, ROI, cost, risk, adoption, roadmap
2. Product and AI	Why agents, trust in AI, accuracy, model choice
3. Required questions	Security, guardrails, RAG, unit testing
4. Technical and delivery	Architecture, scale, data, operations
5. Quick facts	Numbers to have ready

1. CXO and business questions

Q1. What problem does Zylo Reach solve, in one sentence?

Brands overspend on influencers by picking big accounts by gut feel; Zylo Reach picks creators whose audience and content actually fit the product, shows what they should cost, and builds the best plan for a budget, with a reason for every choice.

Q2. What is the business value? Show me the return.

On the same budget, the plan Zylo Reach builds is compared with booking the biggest accounts in the category. In our demo campaigns (synthetic data) that comparison showed roughly 2.5x to 4x the expected orders, 4x to 8x the views from the target audience, and around 50% to 70% lower cost per 1,000 views. The rate check also flags creators asking above market rate and estimates the saving from negotiating them down (thousands of pounds per campaign in the demo).

These are labelled estimates from synthetic data; a pilot with real creators and tracked sales is how we would prove them.

Q3. How is this different from existing influencer platforms?

- **Explainable by design:** every creator has a score breakdown and every exclusion has a reason code.
- **Budget intelligence:** fair market rate per creator, value flags and a budget plan, not just a search directory.
- **Collections:** promote a whole range (Marvel & DC, Disney, Summer Sale) and only keep creators who already post about it.
- **Open by default:** it speaks MCP both ways, so other AI tools can use it and outside data providers can plug in.
- **Connected to commerce:** it reads the same product, stock and price data the shop sells from.

Q4. What does it cost to run?

Very little. Matching, ranking and ROI maths are plain Python and use no AI tokens. Only two things call a language model: the short explanation of each campaign and the chat assistant. A campaign run used about 3,700 tokens in testing, which is fractions of a cent on a budget model and zero on the free or local models we support. Spend is visible in a live token and cost panel, capped by rate limits, and can be hard-capped with a daily token budget.

Q5. Can we trust the AI not to make expensive mistakes?

Yes, because the AI does not make the decisions. Who is eligible, how they rank and what goes into the budget plan are deterministic rules that give the same answer every time. The model only writes the explanation, and the numbers shown on screen come from the engine, not from the model. If the model fails or is busy, the app falls back to rules and labels the result for review.

Q6. What are the main risks?

- **Data quality:** results are only as good as creator data; the demo is synthetic. Mitigation: a pluggable data provider and an MCP connector for licensed data sources.
- **Brand safety:** flagged creators are always excluded, but real-world screening needs a verified data source.
- **Access control:** the back office has no login yet; that is the first item before any real deployment.
- **Over-reliance on estimates:** ROI figures are clearly labelled estimates; a pilot is needed to calibrate them.

Q7. How quickly can a marketing team use it?

A campaign takes about a minute: pick a product or collection (the request fills itself in), review the brief, launch. Results, analytics, the ROI plan and the agent graph are ready as soon as the run finishes. No training data or setup is needed beyond connecting creator data.

Q8. How do we measure success after launch?

- Cost per order and return on ad spend versus previous influencer campaigns.
- Share of budget going to good-value versus above-market creators.
- Time from brief to shortlist (hours today, about a minute here).
- Creator hit rate: share of booked creators who meet their expected views and orders.

Q9. What about compliance, privacy and advertising rules?

Matching uses aggregated audience statistics (age and gender share of followers), not personal data about individual followers, and never uses a creator's own personal attributes. The app stores no customer personal data beyond demo orders. Advertising disclosure rules (ASA in the UK, FTC in the US) apply at contracting stage, which is out of scope for this release and on the roadmap.

Q10. Are we locked into one AI vendor?

No. Models are switched in Settings: OpenRouter, Google Gemini, OpenAI, Anthropic, local Ollama or any LiteLLM-compatible endpoint. The app also runs fully offline with rule-based explanations. Prices per model are editable, so cost reporting follows the choice.

Q11. What would it take to go to production?

- Login, roles and audit trail for the back office.
- Real creator data through a licensed provider (the MCP client and data provider interface are ready).
- PostgreSQL and a job queue for many concurrent runs; encrypted secret storage.
- A 4 to 6 week pilot on two or three campaigns with tracked sales to calibrate the ROI model.

Q12. What is on the roadmap?

- Outcome tracking that tunes the ranking weights from real campaign results.
- Outreach and contracting workflow with human approval.
- Creator content search (captions and transcripts) to sharpen content fit.
- Multi-brand workspaces and budget optimisation across several campaigns.

2. Product and AI questions

Q13. Why use AI agents at all if the decisions are rules?

Agents orchestrate and explain. The Matching Coordinator agent runs seven tools in a fixed order and records every step, which gives the live agent graph and full traceability. The Insights agent turns the numbers into a recommendation a marketer can act on. The chat assistant lets people ask open questions ("which products are low on stock and which micro-creators suit the cheapest one?") and picks the right tools itself.

Q14. How do you know the explanation is accurate?

The model only receives the engine's actual results and is instructed not to invent figures. Cards and tables under each answer are built from tool results, not from model text. Every model turn is recorded in the agent brain, so anyone can see what it saw and said. Limitation: the wording of the explanation itself is not machine-checked against the numbers.

Q15. What happens when the model is slow or down?

Busy or rate-limited calls are retried up to three times with back-off, visibly. After that, campaign explanations use a fixed template and the chat uses a rule-based router that calls the same tools. The result is labelled Needs Review, with a one-click retry.

Q16. Why did you merge the Brief agent into the workflow?

It made one tool call that a rules-based parser does instantly and for free. Removing it cut a model call from every campaign and made brief preparation deterministic; the marketer still reviews the brief before anything runs.

Q17. What is MCP and why does it matter here?

The Model Context Protocol is an open standard for AI tools. Zylo Reach is an MCP server (other AI assistants can query the catalogue, creators and campaigns, with write tools locked by default) and an MCP client (it pulls outside creator data from a provider during each run). Every inbound and outbound call is logged.

3. Required questions

Q18. Is there any security measure implemented, and how are you testing the security measures?

Yes. Security is enforced in the back office and its API, and each measure has automated tests.

Measure	What is in place	How it is tested
Rate limits	Per client: chat 20/min and 300/day, campaign runs 6/min and 60/day, connection tests 10/min, max 3 runs in flight. Over a limit: HTTP 429 with Retry-After. Spoofable X-Forwarded-For is ignored.	test_security.py: per-minute, per-day, per-client, spoofed header and in-flight tests
Token budget	Optional daily cap on model tokens; rule-based work keeps running.	Budget test blocks model calls and allows rules
Security headers	Strict Content-Security-Policy (scripts from this server plus one inline script allowed by hash), nosniff, no framing, strict referrer and permissions policy, no-store on API.	Header tests, plus a test that the CSP hash matches the served page (it caught a real line-ending bug)
Model tokens	Write-only, masked in responses, logs and errors.	test_settings.py: token never returned, even on failed connection tests
MCP access	Write tools off unless switched on with a token; bearer token; DNS-rebinding protection.	test_mcp.py: write gate, token, foreign host header
Uploads and input	Image type checked by file signature (no SVG), size limits, strict schemas.	Upload, validation, oversized and malformed input tests
Attack probes	Path traversal, SQL injection and script payloads.	Parametrised attack tests: no file leaks, injection treated as text, scripts stored as text and served as JSON
Dependencies	pip-audit and npm audit; vulnerable starlette and google-adk upgraded and pinned.	scripts/security-audit runs both scans plus the security tests; currently 0 known vulnerabilities

Not yet in place: login and roles for the back office, encryption of the stored model token, and a manual penetration test (the automated attack tests are a baseline, not a substitute).

Q19. Is there any guardrail implemented, and how are you testing the guardrails?

Yes. The guardrails keep the AI from deciding, inventing or acting on its own.

Guardrail	How it is tested
The model never decides who is shortlisted or ranked; Python does	Ranking with the model on equals the pure engine; same input always gives the same output
Brand-safety-flagged creators always excluded; missing data cannot raise a score; followers are a threshold, not a weight	Engine tests for each rule
Collection campaigns keep only creators who post about the collection's themes	Collection test checks every ranked creator
Chat can read but not change stock or products; launches a campaign only when explicitly asked	Tool list test; preview question saves nothing; explicit launch starts a run
Unclear requests get a question, not a guess	Unclear request and ambiguous brief tests
Result cards built from tool results, never model text; model text never rendered as HTML	Card-source test; HTML injection test in the frontend
Honest failure: retries, then rule or template fallback labelled Needs Review	Overload, permanent error and fallback tests
Spend limits and labelled estimates	Rate limit and budget tests; ROI label and assumptions test

Not yet covered: adversarial prompt-injection tests against hostile creator bios, and automatic checking of the explanation's wording against the numbers.

Q20. Do we need to introduce RAG, or can the existing agentic architecture handle it?

The existing architecture handles it; RAG is not needed today. Zylo Reach's data is structured (audience shares, metrics, themes, stock, prices). The agents get exactly what they need by calling tools that return real, current data, which is more reliable than retrieving text chunks, and exact filters give an explainable reason for every exclusion.

RAG becomes worthwhile when we add long free text: creator captions and video transcripts, brand guidelines, past campaign reports or advertising-disclosure policies. It would be added as one more tool (for example search_creator_content) feeding a signal into content fit and quoting sources in explanations, while thresholds, ranking and brand safety stay deterministic. The likeliest first use is creator captions and transcripts.

Q21. What kind of unit testing was done?

Automated unit and integration tests run on every change: 110 backend test cases (pytest; 97 test functions, some parametrised) and 21 frontend tests (Vitest and Testing Library). All pass on the latest run.

Suite	Tests	What it covers
test_engine.py	12	Level 1 thresholds, missing data goes to review, brand safety, audience alignment, weights and score normalisation, stable ranking order, determinism, follower count not dominant
test_agents.py	9	Brief preparation without a model, collection detection, the seven coordinator steps, token reporting, agent brain, collection theme filter, outside creators fetched over a real MCP connection
test_api.py	13	Full campaign workflow, threshold re-runs, zero results, determinism, collections, ROI within budget, usage and prices, analytics with outside creators (regression)
test_chat.py	18	Routing to the right agents, preview vs launch, real ADK loop with a scripted model, retries, fallbacks, validation, cards from results
test_mcp.py	14	Tool surface, write gate, token, host checks, activity log and secret masking
test_security.py	16 (23 cases)	Headers and CSP, rate limits, token budget, traversal, injection, payloads, method checks
test_inventory.py	9	Catalogue, validation, atomic stock, uploads, migrations
test_settings.py	6	Token masking, offline mode, connection tests
Frontend (5 files)	21	Assistant text rendering and HTML safety, agent brain, token usage dropdown, floating assistant, 'Why' cell, collapsible panels, branding, formatting, shared UI

How it is done: an in-memory database per test run, a scripted fake model so agent tests run offline and predictably, a real MCP subprocess for the outside-data test, and one command (scripts/security-audit) for dependency scans plus security tests. Beyond unit tests we ran browser checks: an overflow audit of 14 pages at six screen widths and a check that every agent graph label fits.

4. Technical and delivery questions

Q22. What is the architecture?

A React single-page app talks to one FastAPI backend that also hosts the MCP server. The Matching Coordinator agent runs seven deterministic Python tools (resolve target, load creators, fetch outside creators over MCP, Level 1, Level 2, ranking, ROI plan); the Insights agent explains. Data lives in SQLite (portable to PostgreSQL) and an in-memory creator dataset.

Q23. How does the matching algorithm work?

- **Level 1, eligibility:** at least 1,000 followers and 500 median promotional views (editable), clear brand safety; missing metrics go to review.
- **Level 2, fit:** audience fit (age 75%, gender 25%) averaged with content fit; must reach 55. Collections also require their content themes.
- **Ranking:** weighted score: audience 25, content 25, engagement 15, promotional views 15, past collaborations 10, brand affinity 5, region 5.

Q24. Will it scale?

A run over 1,200 creators takes about two seconds of compute. For larger catalogues and many users we would move to PostgreSQL, run campaigns on a job queue and cache creator data; the engine itself is vectorisable and stateless.

Q25. How do you observe it in production?

Every run stores a trace and agent brain, the agent graph replays it, the MCP activity log records inbound and outbound calls, and the header shows live token usage and cost by source, model and agent.

Q26. What tech stack did you use?

React 19, TypeScript, Vite, Tailwind CSS, Framer Motion and D3 on the frontend; Python, FastAPI, Pydantic and SQLAlchemy on the backend; Google ADK with LiteLLM for agents; the official MCP SDK; pytest and Vitest for testing.

Q27. What was the hardest problem?

Keeping AI useful without letting it decide. We drew a hard line: the model explains and chooses tools, Python decides. That made the system explainable, testable and cheap, and let it keep working when free models were overloaded.

5. Quick facts to have ready

Item	Figure
Creators evaluated per run	1,200 own + outside creators over MCP (29 for a Marvel & DC run)
Products and collections	73 products, 5 collections
Model calls per campaign	1 (the explanation); matching uses none
Tokens per campaign (tested)	About 3,700
Tests	110 backend cases and 21 frontend tests, all passing
Known dependency vulnerabilities	0 (pip-audit and npm audit)
Demo ROI comparison	About 2.5x to 4x expected orders vs biggest accounts, same budget (synthetic, estimate)
Data	Synthetic demonstration data throughout

All ROI and reach figures are estimates from synthetic data and are labelled as such in the product.